
AutoResearch for Traffic Congestion Prediction: Segment-Relative Features Break a Labeling-Driven F1 Ceiling

Sherry Huang | STAT 390 Capstone | Northwestern University |
xiyuhuang2026@u.northwestern.edu

Code: github.com/SherryHuang0311/traffic-autoresearch

Abstract. We present an AutoResearch loop applied to 30-minute-ahead congestion prediction on Chicago road segments (786K observations, Sept. 2023). Over 30 experiments we identify and partially fix a structural labeling problem: the global congestion threshold (~23 mph) mislabels ~16% of training rows because different road segments have very different normal speed ranges. Adding four segment-relative features breaks an F1 ceiling at 0.658 that no amount of hyperparameter tuning could cross. LightGBM with full feature subsampling achieves the final validation F1 = **0.6806** (+21.6% over logistic regression baseline), stable across random seeds (mean 0.674, std 0.005). A one-time evaluation on the held-out test set yields **F1 = 0.6183** (+10.5% above the baseline), with the val→test gap explained by a distribution shift in positive rate (29.7% → 24.4%).

1 Introduction

Traffic congestion prediction from speed history alone — without real-time feeds or external map data — is a constrained but practically relevant problem. This project formalizes it as an AutoResearch contract: predict whether a given Chicago road segment will be congested exactly 30 minutes from now, evaluated by validation binary F1 on a fixed time-based split. The AutoResearch loop modifies only **src/model.py** (model and features) while **src/run.py** (data split, label definition, metric) is frozen, guaranteeing all 30 experiments are evaluated identically.

2 Data, Metric, and Baseline

Dataset. Chicago Traffic Tracker (City of Chicago Open Data, Sept. 2023). 786,420 observations. Each row is one speed reading for one road segment at a 10-minute timestamp. **SEGMENT_ID** identifies individual road segments.

Label. A segment is congested if speed < 30th percentile of all training speeds (~23 mph). Class ratio 3.49:1 (77.7% non-congested). **Target:** congestion status 3 steps (30 min) ahead via shift(-3) per segment.

Split. Deterministic, time-based: train 70%, validation 15%, test 15% (locked — never accessed during development).

Features (17 total). SPEED, lags 1–6, rolling_mean_3, rolling_std_3, speed_diff, HOUR, DAY_OF_WEEK, MONTH, and four segment-relative features (segment_mean_speed, segment_std_speed, speed_vs_seg_mean, speed_zscore) computed from training data only.

Baseline. Logistic regression, lags 1–3, time features: F1 = 0.5598, P = 0.755, R = 0.445 (exp_001).

3 AutoResearch Loop Design

Each iteration: (1) read model.py; (2) propose one change; (3) edit model.py; (4) run python src/run.py "description"; (5) if F1 improves → commit; else → git checkout src/model.py and log status=discard. Results append to experiments/results.csv. The loop is greedy (single-step revert, no lookahead). The frozen evaluator prevents metric gaming — changing the split or threshold after seeing a good result is structurally impossible.

4 Experiments and Results

4.1 F1 Progression by Phase

Phase	Key change	Best F1	Gain
Baseline (exp_001)	Logistic regression	0.5598	—
Model selection (exp_002–007)	RF + class balance + lags 1–6	0.6566	+0.097
HP grid (exp_008–013)	Depth, trees, undersampling ratio	0.6585	+0.002
Threshold sweep (exp_014–018)	Decision threshold 0.35–0.50	0.6585	+0.000
Segment features (exp_019–021)	speed_zscore, speed_vs_seg_mean, seg. mean/std	0.6727	+0.014
Model comparison (exp_022–025)	LightGBM vs RF vs XGBoost vs HGB	0.6780	+0.005
LightGBM tuning (exp_026–030)	num_leaves, subsampling, n_estimators	0.6806	+0.003

Table 1: F1 progression across 30 experiments (0 crashes). Each phase builds on the previous best configuration.

4.2 The Ceiling Discovery

Experiments 3–13 confirmed a hard ceiling at $F1 \approx 0.658$. Varying `n_estimators` (50–200), `max_depth` (4–12), and undersampling ratio (1:1–2:1) moved F1 by under 0.003 total. The root cause: the global 23 mph threshold mislabels ~16.3% of training rows. A highway segment averaging 45 mph is labeled 'congested' at 22 mph even if that is its normal rush-hour pace. The fix — adding segment-relative features computed from training data only via `groupby('SEGMENT_ID')['SPEED'].agg(...)` — gives the model road-specific context. F1 jumped 0.014 in one step.

4.3 Model Comparison and Final Tuning

Model (exp)	F1	Precision	Recall
Random Forest (exp_020)	0.6727	0.644	0.704
HistGradientBoosting (exp_022)	0.6553	0.540	0.834
XGBoost (exp_023)	0.6712	0.625	0.724
LightGBM 17 features (exp_025)	0.6780	0.638	0.724
LightGBM tuned (exp_030, final)	0.6806	0.635	0.733

Table 2: Model comparison (identical setup: 17 features, 2:1 undersample, threshold=0.40). Red = discard. Green = final best. LightGBM consistently edged Random Forest. Reducing `num_leaves` (63→31), setting `min_child_samples=10`, and adding 20% feature and row subsampling (`colsample_bytree=0.8`, `subsample=0.8`) gave +0.003 F1. Increasing `n_estimators` to 500 gave a further marginal gain.

4.4 Stability

Seed	F1	Precision	Recall
42	0.6709	0.5953	0.7685
0	0.6684	0.5998	0.7547
123	0.6735	0.5998	0.7678
7	0.6821	0.6357	0.7358
99	0.6740	0.6212	0.7366
Mean ± Std	0.674 ± 0.005	0.610 ± 0.016	0.753 ± 0.014

Table 3: Stability across 5 random seeds. Run.py result (0.6806, seed=42) is within 1.5 std of the mean.

5 What Worked and What Did Not

Worked. The single most impactful step was adding segment-relative features (+0.014 F1). LightGBM marginally outperformed Random Forest (+0.003). Undersampling 2:1 achieved better F1 than `class_weight='balanced'` while maintaining higher precision. The frozen evaluator made all 30 comparisons valid.

Did not work. HistGradientBoosting underperformed in both attempts. Threshold tuning below 0.40 reliably raised recall but dropped F1. Six hyperparameter experiments on RF moved F1 by under 0.003 total.

Fundamental limit. The congestion label is still globally defined. Fixing this requires changing the frozen `run.py` — the residual label noise is irreducible within the current evaluation framework.

6 Conclusion

Split	F1	Precision	Recall
Baseline validation (exp_001, LR)	0.5598	0.7549	0.4448
Validation (exp_030, LightGBM)	0.6806	0.6352	0.7329
Test (final, one-time)	0.6183	0.5639	0.6843
Val → Test gap	−0.0623	−0.0713	−0.0486

Table 4: Final results. Green = best validation. Blue = test (one-time, never seen during development). Val→Test gap explained by distribution shift: test period positive rate 24.4% vs 29.7% in validation.

The clearest lesson: for structured tabular data, feature engineering that encodes domain knowledge (road-specific speed norms) is more impactful than model selection or hyperparameter search. The loop arrived at this insight through systematic elimination — every failed tuning experiment narrowed the search space. Test F1 = **0.6183** confirms generalization: +10.5% above the baseline, with no tuning performed after the test set was opened.

Reflection Memo — Agent Performance and Human-Agent Collaboration

What the Agent Did Well

Systematic elimination with zero bookkeeping errors. Over 30 experiments the agent never conflated results, never ran a worse model forward, and never lost track of the current best. Every discard was caught and reverted correctly. This kind of precise, patient bookkeeping is where the agent genuinely outperformed informal human iteration.

Identifying the labeling problem. The agent measured the 16.3% label disagreement between the global threshold and a segment-relative threshold, connected it to the ceiling at $F1=0.658$, and designed the fix — segment features from training data only with explicit leakage analysis. A human researcher might have spent weeks tuning before arriving at this diagnosis.

Execution speed. 30 experiments with full git discipline ran in hours without fatigue or attention lapses.

What the Agent Did Poorly

Greedy search with no lookahead. The loop committed to Random Forest for 20 experiments before trying LightGBM — which turned out to be marginally better. A simple early model tournament would have found LightGBM sooner.

No repeat-on-change logic. HistGradientBoosting was dropped after exp_005 (without segment features) and not retried until exp_022 — a 17-experiment delay. The loop had no mechanism to flag experiments for retry under changed conditions.

Single-seed evaluation. Every experiment was a single run (seed=42). Keep/discard decisions were made on single-point estimates, risking lucky-run false positives. Multi-seed evaluation should have been standard.

What Required Human Judgment

Ceiling diagnosis. Measuring the 16.3% label disagreement was automatic; understanding that this was the *fundamental limiting factor* — that no model could overcome systematically wrong labels — required framing of the learning problem.

Scope lock. Deciding to stop exploring new model families and commit to LightGBM tuning was a judgment about diminishing returns that the agent cannot make. The agent measures what improved; humans judge when further improvement is not worth the cost.

Presentation narrative. Which experiments to highlight — not just 'highest F1' but 'most interesting finding' — is a human judgment about what constitutes a contribution worth communicating.

How Would You Redesign the Loop

- **Early model tournament:** in the first 10 experiments, try every major model family on the baseline feature set. Lock the winner, then tune.
- **Repeat-on-change rule:** when a structural change is made (new features), automatically re-run previously discarded configurations under the new conditions.
- **Multi-seed evaluation:** run each experiment with 3 seeds, report the median. Costs 3× compute but eliminates lucky-run false positives.
- **Automatic ceiling diagnosis:** after every 5 non-improving experiments, compute label agreement between the current strategy and alternatives and surface it as a diagnostic.

Final Reflection: Research With AI Agents

The agent is excellent at *execution within a well-defined protocol* and poor at *knowing when to change the protocol*. The most valuable human contribution was not running experiments — the agent did that reliably. It was asking the right diagnostic question: *why is the ceiling there?*

Agent-assisted research shifts the bottleneck from execution to diagnosis. The agent can run 100 experiments in the time it would take a human to run 5 — but if those 100 experiments are all asking the wrong question, the agent produces 100 wrong answers very efficiently. The human's job becomes: read results fast, spot the pattern the agent is missing, and redirect.

In this project that happened once — Week 5, when the ceiling pattern was identified and the labeling problem diagnosed. That one redirection was worth more than all 20 hyperparameter-tuning experiments combined.